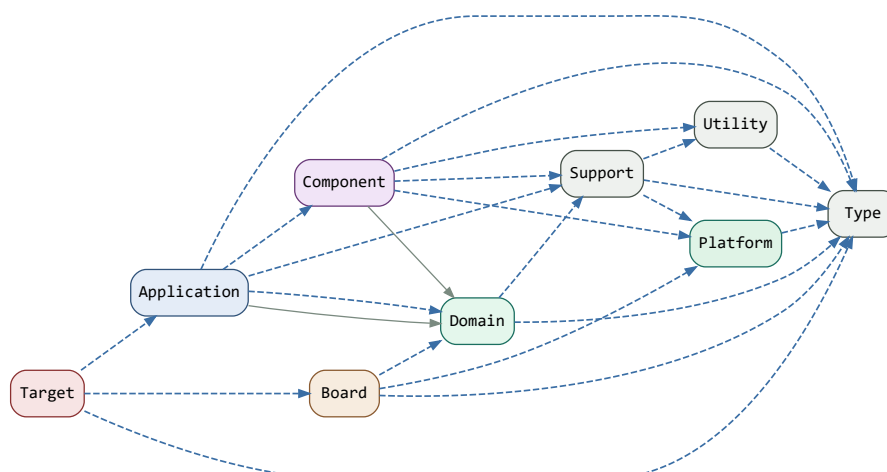


AstraLEO Explore SAMV71 Peripherals, etc.

Build using GCC and C++20. Using FreeRTOS (FreeRTOSv202604.01-LTS).

Top-level modules

- **Application/** — the app itself: `Hello_world` (composition root, wired up in `main()`), `Provider` (the data providers it routes requests to) and `Support` (C helper code, e.g. the gamma peak detector and isotope table).
- **Board/** — board support packages, one per physical board (currently `V71_EK`), wiring together the platform, components and pin table for that board.
- **Component/** — drivers for discrete hardware parts (`IDE3380`, `SiPM_bias`, `STTS22H`), independent of any one board.
- **Domain/** — application-domain types and the generated command/keyword dictionary (`Domain/Generated_code`) that ties the console/CLI layer to those types.
- **Platform/** — MCU- and RTOS-facing code: `SamV71_platform` (and its `SAMV71Q21B` startup/SDK layer), `Common_platform` and `FreeRTOS_platform`.
- **Support/** — cross-cutting infrastructure: the console service, the CLI parser and instruction dispatch (`Plumbing/`), and the configuration/attribute repository.
- **Target/** — top-level build targets that assemble an application, board and platform into one firmware image (currently `V71_EK_hello_world`).
- **Type/** — shared type definitions: abstract hardware interfaces (`Abstract/`), basic value/status types (`Basic/`) and generic transfer-request types (`Generic/`).
- **Utility/** — small, dependency-free helpers (ring buffer, simple string/math) used throughout the rest of the tree.

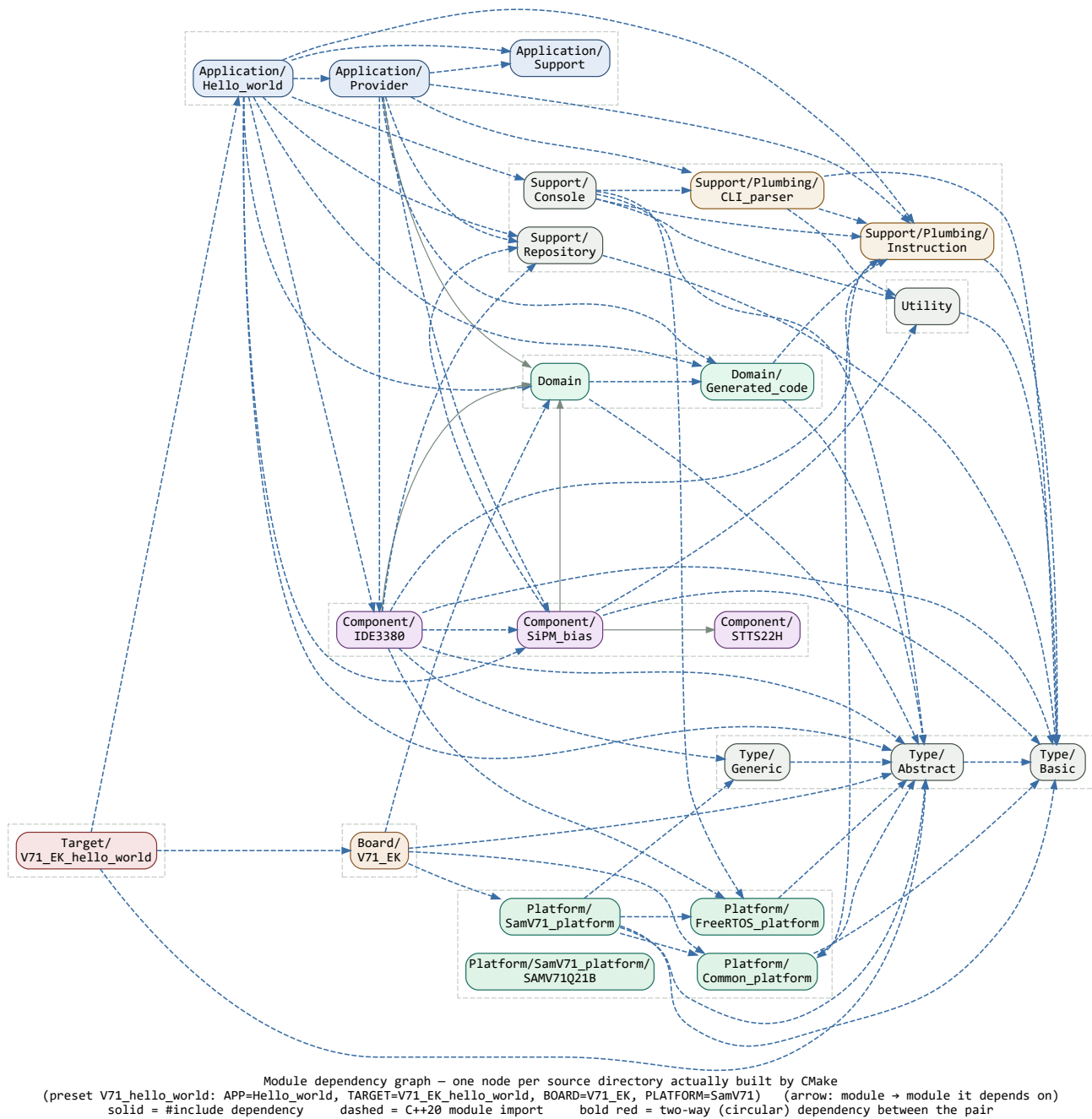


Top-level module dependency graph -- one node per top-level `src/` directory
reset V71_hello_world: APP=Hello_world, TARGET=V71_EK_hello_world, BOARD=V71_EK, PLATFORM=SamV71) (arrow: directory -> directory it depends on
solid = #include dependency dashed = C++20 module import bold red = two-way (circular) dependency between the pair
Rolled up from `doc/module_dependencies.svg` (per-source-directory graph).

The graph above collapses every source directory into its top-level `src/` area (`Application/`, `Board/`, `Component/`, `Domain/`, `Platform/`, `Support/`, `Target/`, `Type/`, `Utility/`). The full, per-

source-directory graph it's rolled up from:

Dependencies



Wiring

Aggregation graphs of the two concrete objects wired together in `main()`
(`V71_EK_hello_world.cpp`):

What `Hello_world` (the application) owns and references:

Command	Description
AT+DEMO_CADENCE=<seconds(int 0..1000000)>	Set telemetry interval (s) in Demonstration Mode.
AT+DEMO_CHANNEL=<channel(int 0..18)>	Set channel to be used in live view demo. 0=inhibit readout, 1..16=input channel, 17=analog summing, 18=digital summing.
AT+IDLE_CADENCE=<seconds(int 0..1000000)>	Set telemetry interval (s) in Idle Mode.
AT+MODE_DEMO	Set demonstration mode.
AT+MODE_IDLE	Set idle mode.
AT+MODE_NOMINAL	Set nominal mode.

Command	Description
AT+NOMINAL_ALARM= <micro_sievert(int 0..1000000)>	Set alarm threshold (uSv/h).
AT+NOMINAL_CADENCE=<seconds(int 0..1000000)>	Set telemetry interval (s) in Nominal Mode.

Provider: Spectroscopic_data

Command	Description
AT+FORMAT_CPS	Send event counters.
AT+FORMAT_N42	Use complex histogram format (N42).
AT+FORMAT_R6	Use simple histogram format (R6).
AT+GET_RESET	Send the histogram and clear the buffers.
AT+GET	Send the histogram.
AT+TIME=<date(int 0..0xFFFFFFFF)>,<time(int 0..1000000)>	Set date and time.

Provider: Instrument_calibration

Command	Description
AT+CAL_ADC_V35=<cal_35V(int -100000..-1)>	Set bias ADC calibration 35V setpoint.
AT+CAL_ADC_V40=<cal_40V(int -100000..-1)>	Set bias ADC calibration 40V setpoint.
AT+CAL_DAC_V35=<cal_35V(int -100000..-1)>	Set bias DAC calibration 35V setpoint.
AT+CAL_DAC_V40=<cal_40V(int -100000..-1)>	Set bias DAC calibration 40V setpoint.
AT+CAL_DIAG	Misc. diagnostic information wrt. calibration.
AT+CAL_TEST_BIAS	Find common bias voltage using dark count rate.
AT+CAL_TEST_NOISE	Find channel noise floor using threshold scan.
AT+CAL_TEST_OFFSET	Find channel offset voltage using fixed source.
AT+CAL_TEST_PEDESTAL	Find channel pedestal using forced readout.
AT+CAL_TRACE	Toggle diagnostic trace on/off.

Provider: Configuration_manager

Command	Description
AT+ASIC_DUMP	Dump all the ASIC registers.
AT+ASIC_LOAD	Update all ASIC registers from current configuration.
AT+ASIC_REG=<reg_addr(int 0..32)>,<data32(int 0..0xFFFFFFFF)>	Write the ASIC register.
AT+ASIC_REG=<reg_addr(int 0..32)>	Read the ASIC register.

Command	Description
AT+CONFIG_APPLY	Apply the current configuration.
AT+CONFIG_CLEAN	Clean the persistent storage and set the default configuration.
AT+CONFIG_DUMP	Dump the current configuration.
AT+CONFIG_LOAD	Load the current configuration from the persistent storage.
AT+CONFIG=<offset(int 0..4095)>,<data32(int 0..0xFFFFFFFF)>	Set configuration attribute value.
AT+CONFIG=<offset(int 0..4095)>	Get configuration attribute value.
AT+CONFIG_SAVE	Save the current configuration to the persistent storage.

Provider: Housekeeping

Command	Description
AT+DIAG	Send miscellaneous diagnostic information.
AT+HELP	Show commands.
AT+STATUS	Send mode and other system data.
AT+TEST	Initiate a self-test sequence.
AT+TRACE	Toggle trace on/off.
AT+VERSION	Send version and other build information.

▼ Keywords (SpectraNode version 1)

Id	Name	Description
0	"No key"	Special purpose
1	ADC	
2	ALARM	
3	APPLY	
4	ASIC	
5	BIAS	
6	CADENCE	
7	CAL	
8	cal_35V	
9	cal_40V	
10	channel	
11	CHANNEL	

Id	Name	Description
12	CLEAN	
13	CONFIG	
14	Configuration_manager	
15	CPS	
16	CSV	
17	DAC	
18	data16	
19	data32	
20	data8	
21	date	
22	DEMO	
23	DIAG	
24	DUMP	
25	FORMAT	
26	gain	
27	GET	
28	HELP	
29	Housekeeping	
30	IDLE	
31	INPUT	
32	Instrument_calibration	
33	LOAD	
34	micro_sievert	
35	millis	
36	MODE	
37	Mode_control	
38	N42	
39	NOISE	
40	NOMINAL	
41	offset	
42	OFFSET	
43	PEDESTAL	
44	R6	

Id	Name	Description
45	REG	
46	reg_addr	
47	RESET	
48	SAVE	
49	seconds	
50	Spectroscopic_data	
51	STATUS	
52	TEST	
53	time	
54	TIME	
55	TRACE	
56	V35	
57	V40	
58	VBIAS	
59	VERSION	
255	"Wildcard"	Special purpose, used in search

▼ Structure (SpectraNode_structure_definition)

Identifier	Type	Default	Description
------------	------	---------	-------------

(Empty — the structure definition is currently commented out in its source XML and isn't part of the build.)

C++20 modules

Several parts of `src/` (`Type/` , `Support/` , `Application/` , `Component/` , `Board/V71_EK` , the built `Platform/` directories, `Utility/` and `Domain/Generated_code`) are built as named C++20 modules rather than traditional header/source pairs. A quick primer:

- **Module interface unit** — declares and exports the module's public API. File extension is `.cppm` :

```

export module Support.Console_service;    // module name (dots are just part of the
name)

import Type.Abstract_service;              // pull in another module

export class Console_service : public Abstract_service { // exported: visible to
importers
    ...
};

void log_internal(...);                    // not exported: module-private

```

- **Module implementation unit** — provides definitions for a module declared elsewhere, without re-exporting its interface: `module Support.Console_service;` (no `export`) at the top of a `.cpp` file. Counts as depending on whatever module it implements.
- **Importing a module** — `import Some.Module.Name;` (or `export import ...;` to re-export it through the importing module) instead of `#include`-ing a header. No preprocessor, no include guards, no macro leakage between translation units.
- **Partitions** — a module can be split across files internally with `export module Foo:Partition_name;` / `import :Partition_name;`; partitions are only visible to other pieces of the same module, never to importers of `Foo`.
- **Naming convention used in this repo** — module names mirror their source directory, dotted (e.g. `Support/Console/Console_service.cppm` exports `Support.Console_service`), which is why the dependency graphs above draw `import` edges (dashed) between the same directory nodes as `#include` edges (solid).

This project still mixes plain headers (mainly SDK/RTOS/third-party code, plus a few C-API sources) with modules, hence the graphs above track both `#include` and `import` edges rather than assuming one or the other.